

Detector de Spam Bilingüe (Inglés + Español)

Documento Técnico — Proyecto Final

Universidad Nacional Curso: Inteligencia Artificial Profesor: Venegas Integrante(s): _____

Fecha de entrega: 8 de junio de 2026

1. Introducción

El correo y la mensajería electrónica son hoy el principal vector de ataques de *phishing*, fraude y publicidad no deseada. El **spam** no solo degrada la experiencia del usuario: es la puerta de entrada a estafas bancarias, robo de credenciales y distribución de malware. Filtrar automáticamente estos mensajes es, por tanto, un problema de seguridad de primer orden.

Este proyecto construye un **agente inteligente** capaz de clasificar mensajes como *spam* o *ham* (legítimo) de forma automática, usando técnicas de Procesamiento de Lenguaje Natural (NLP) y aprendizaje supervisado. A diferencia de los filtros basados en reglas fijas ("si contiene *FREE* entonces *spam*"), nuestro agente **aprende** las regularidades del lenguaje del spam a partir de datos, y generaliza a mensajes nunca vistos.

El sistema es **bilingüe** (inglés y español), una decisión deliberada: los usuarios hispanohablantes reciben campañas de phishing localizadas (BBVA, Santander, SAT, CFE, Mercado Libre) que un modelo entrenado solo en inglés no detecta. El entregable incluye un notebook reproducible, un modelo serializado y un sistema interactivo de predicción.

2. Planteamiento del problema

La detección de spam se formula como un problema de **clasificación binaria supervisada**. El agente aprende una función

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

que mapea el texto crudo de un mensaje a una etiqueta de clase.

	Variable	Tipo	Dominio	Descripción
Entrada $\mathcal{X}$	text	Texto libre	Cadena Unicode de longitud variable (EN/ES)	Contenido crudo del mensaje, con URLs, números y emojis
Salida $\mathcal{Y}$	label	Categórica binaria	spam, ham	Clase predicha
Salida (aux.)	P (spam)	Continua	[0, 1]	Probabilidad estimada de spam

Regla de decisión: se predice spam si $P(\text{spam} \mid \text{texto}) \geq t^*$, donde t^* es un umbral calibrado (no necesariamente 0.5).

Análisis PEAS del agente

Componente	Definición
P — Performance	Accuracy, Precision, Recall y F1. Métrica prioritaria: Recall sobre la clase <i>spam</i> .

Componente	Definición
E — Environment	Mensajes en inglés o español, con HTML, URLs, números y errores ortográficos.
A — Actuators	Etiqueta <i>spam/ham</i> + nivel de confianza $\in [0, 1]$ + probabilidades por clase.
S — Sensors	El texto crudo del mensaje.

Tipo de entorno: totalmente observable, un solo agente, determinístico, episódico y estático. **Tipo de agente:** *model-based* — construye una representación interna (pesos o probabilidades) durante el entrenamiento y la aplica en inferencia.

¿Por qué priorizar Recall? Un falso negativo (spam que llega a la bandeja de entrada) puede derivar en fraude o robo de credenciales; un falso positivo (un mensaje legítimo enviado a la carpeta de spam) es una molestia recuperable. En un contexto de seguridad, el costo asimétrico justifica maximizar la detección de spam aun a costa de bloquear algún mensaje legítimo.

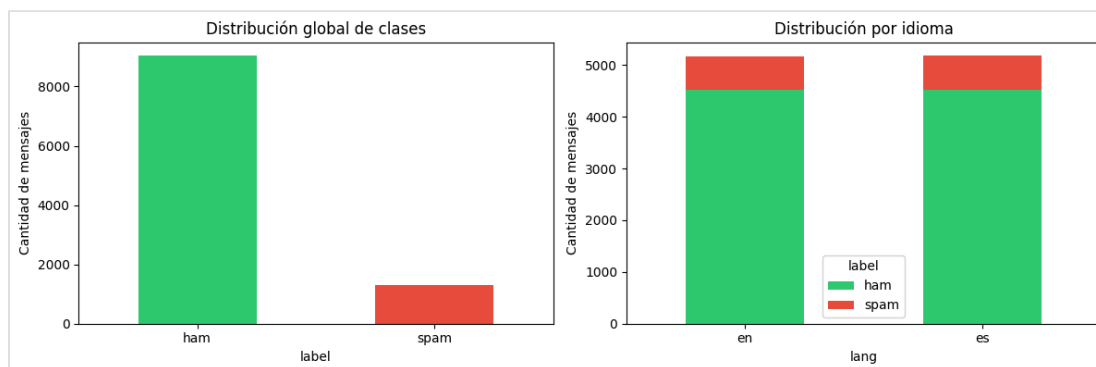
3. Metodología

3.1 Datasets

Se combinaron dos fuentes públicas para construir un corpus bilingüe:

Dataset	Idioma	Fuente	Mensajes
SMS Spam Collection (UCI)	Inglés	Almeida & Hidalgo (2011)	5 574
SMS Spam Multilingual	Español	dbarbedillo (traducción del UCI a 21 idiomas)	5 572
Seed de phishing local	Español	Curado manualmente (BBVA, SAT, CFE, etc.)	61

Tras la unión y la **deduplicación** por texto, el corpus reúne **~10 331 mensajes** con una proporción global de spam del **12.7 %** — un dataset fuertemente **desbalanceado**, fiel a la realidad (la mayoría del correo es legítimo).



3.2 Preprocesamiento

Cada mensaje pasa por una función `clean_text` que:

1. Convierte a minúsculas.
2. Sustituye URLs por el token `__url__` y números por `__num__` (preservando la *señal* sin memorizar valores concretos).
3. Elimina HTML, puntuación y caracteres no informativos.
4. Filtra **stopwords** combinadas de inglés y español (**504** palabras de NLTK).

El reemplazo por *tokens* en lugar de la eliminación es una decisión clave: la presencia de una URL o un número es altamente predictiva de spam, aunque su valor exacto no lo sea.

3.3 Balanceo y partición

Para evitar que el modelo aprenda a predecir siempre *ham*, se aplicó **undersampling** de la clase mayoritaria, llevando el corpus a **3 287 mensajes** con ~40 % de spam. Se dividió de forma **estratificada** en **train (2 629)** y **test (658)**, con `random_state=42` para reproducibilidad. La

proporción de spam se mantiene en ~40 % en ambas particiones, y el test conserva los dos idiomas (337 EN / 321 ES).

3.4 Vectorización y modelos

Se entrenaron y compararon **tres modelos**, cada uno con una estrategia de representación distinta:

#	Modelo	Vectorización	Intuición
1	Naive Bayes Multinomial	Bag of Words	Generativo; cuenta frecuencias de palabra por clase
2	Regresión Logística	TF-IDF de palabras	Discriminativo; pondera palabras por relevancia
3	Regresión Logística	TF-IDF de <i>char n-grams</i> (<code>char_wb</code>)	Robusto a errores ortográficos y agnóstico al idioma

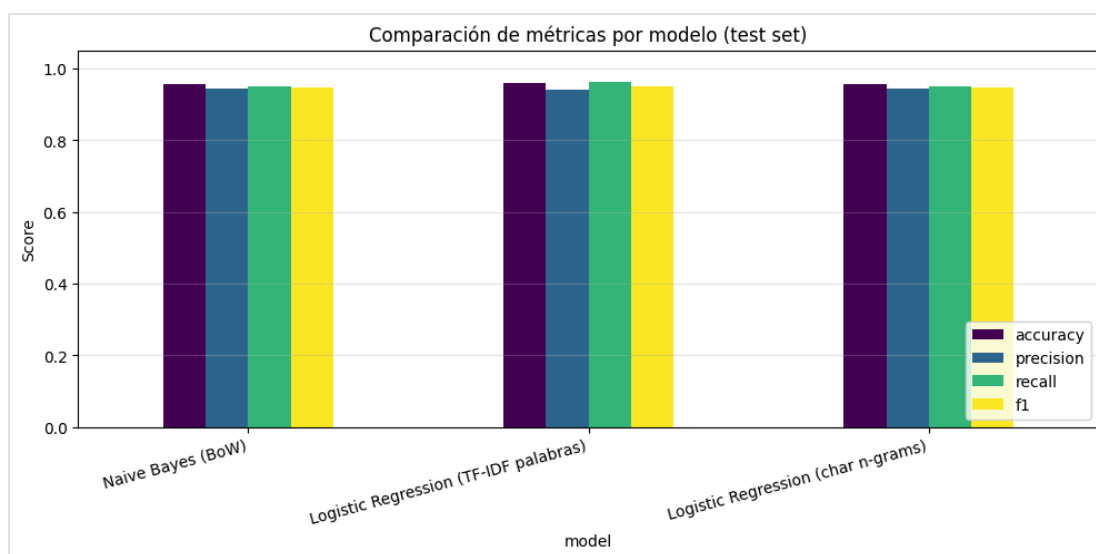
3.5 Validación, tuning y calibración

- **Cross-validation k-fold (k=5)**: se midió la brecha *train* vs *CV* para descartar **overfitting**.
- **GridSearchCV**: búsqueda de hiperparámetros (18 combinaciones × 3 folds) sobre `C`, `min_df` y `ngram_range`.
- **Calibración de threshold**: se eligió el umbral que **maximiza Recall** manteniendo Precision $\geq$ 0.85, en lugar del 0.5 por defecto.

4. Resultados

4.1 Comparación de los tres modelos (test set)

Modelo	Accuracy	Precision	Recall	F1
Naive Bayes (BoW)	0.9574	0.9434	0.9506	0.9470
Regresión Logística (TF-IDF palabras)	0.9605	0.9405	0.9620	0.9511
Regresión Logística (char n-grams)	0.9574	0.9434	0.9506	0.9470



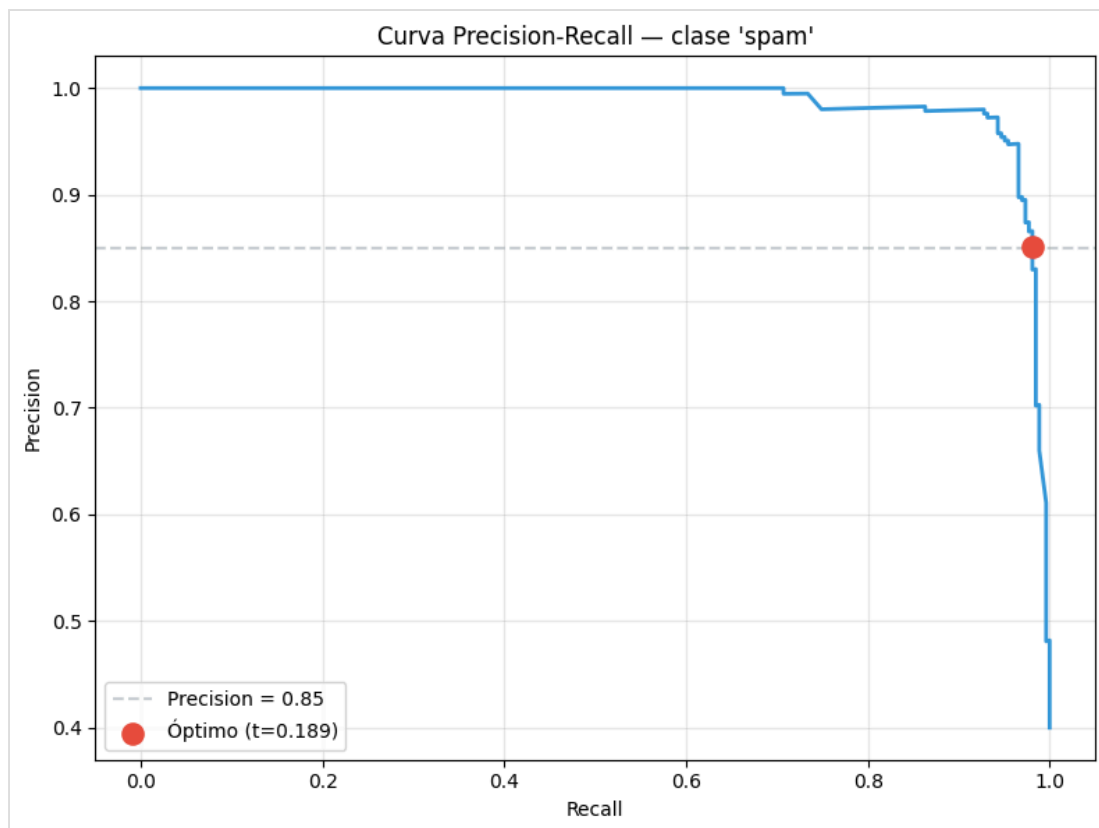
Los tres modelos superan el 95 % de F1. La **Regresión Logística con TF-IDF de palabras** ofrece el mejor Recall (0.962), por lo que se seleccionó como modelo base para el tuning.

4.2 Anti-overfitting y tuning

La cross-validation mostró brechas *train*–CV pequeñas (≈ 0.02 – 0.03 en F1) para los tres modelos, lo que **descarta sobreajuste**. El `GridSearchCV` encontró como mejores hiperparámetros `C = 10.0`, `min_df = 5`, `ngram_range = (1,1)`, alcanzando un **F1 macro en CV de 0.9586**. El modelo final tuneado obtiene en el test set:

Métrica	Valor
Accuracy	0.9620
Precision (spam)	0.9542
Recall (spam)	0.9506
F1 (spam)	0.9524

4.3 Calibración del threshold

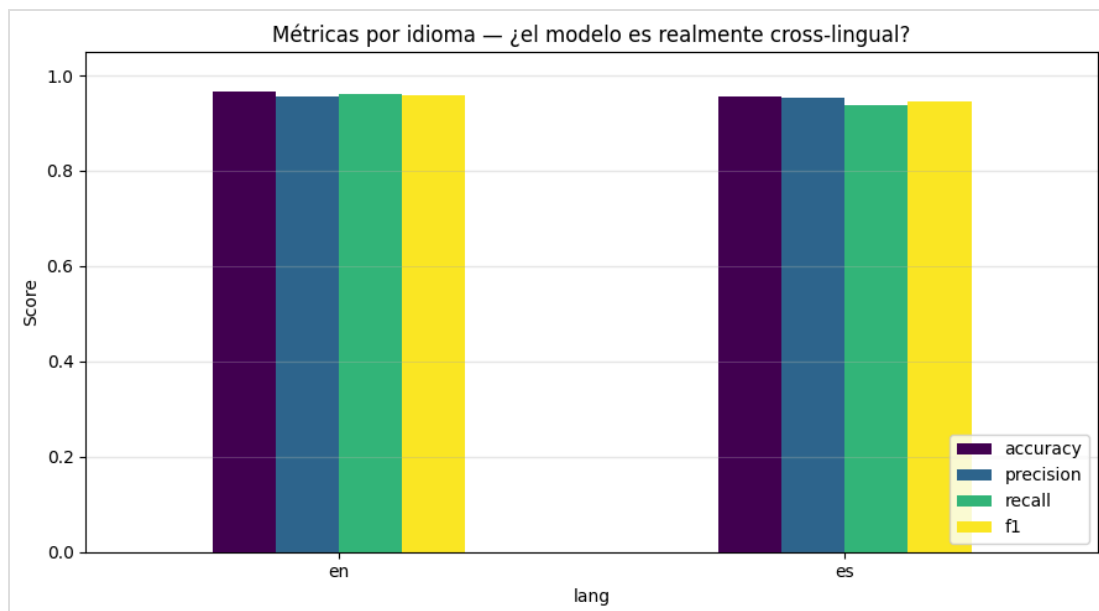


El umbral óptimo resultó $t^* = 0.189$ (frente a 0.5 por defecto). El efecto sobre la clase *spam* es directo:

Threshold	Precision	Recall	F1
0.500 (defecto)	0.9542	0.9506	0.9524
0.189 (óptimo)	0.8515	0.9810	0.9117

Bajar el umbral eleva el Recall del 95.1 % al **98.1 %** — coherente con la prioridad de seguridad — a cambio de una caída tolerable de Precision.

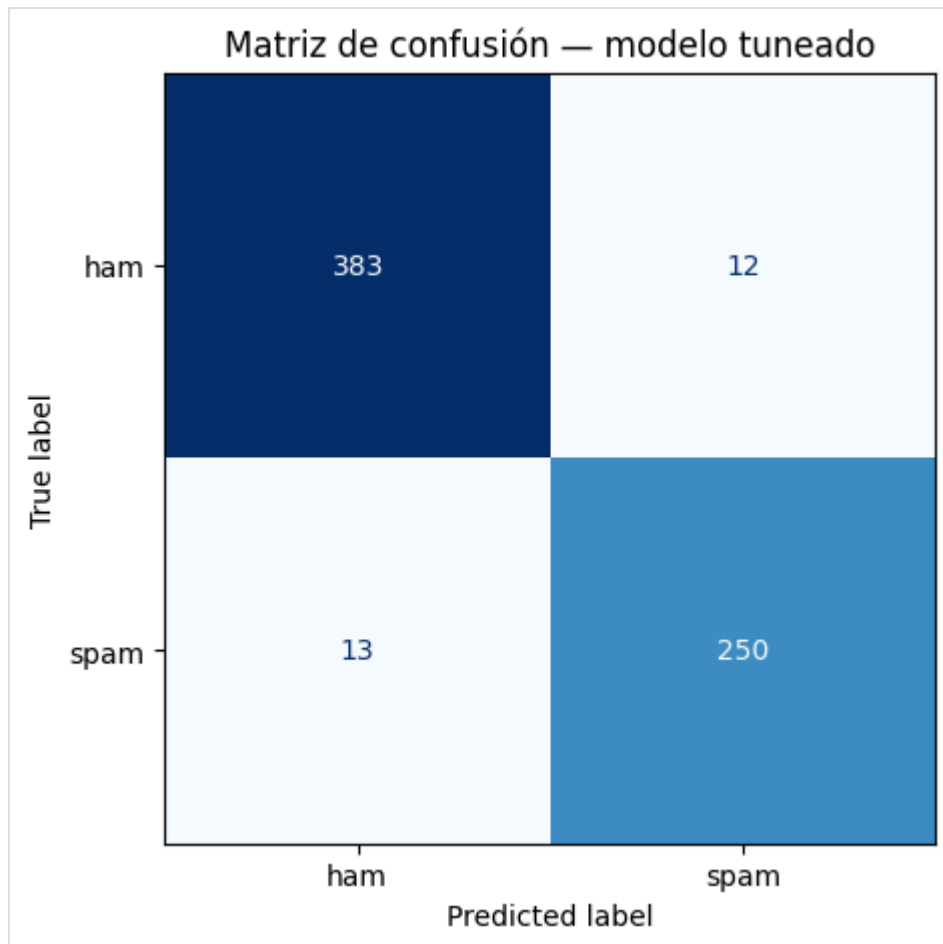
4.4 Evaluación *cross-lingual* (por idioma)



Idioma	n	Accuracy	Precision	Recall	F1
Inglés (EN)	337	0.9674	0.9549	0.9621	0.9585
Español (ES)	321	0.9564	0.9535	0.9389	0.9462

El modelo funciona **de forma consistente en ambos idiomas**, con apenas ~1 punto de diferencia en F1 — evidencia de que el corpus bilingüe y el preprocesamiento con *tokens* generalizan bien entre lenguas.

4.5 Matriz de confusión y análisis de errores



Sobre los 658 mensajes de test: **383** verdaderos negativos, **250** verdaderos positivos, **12** falsos positivos (ham bloqueado) y **13** falsos negativos (spam no detectado). En total **25 errores (3.80 %)**. Al inspeccionar los errores, los falsos negativos corresponden a mensajes muy cortos o ambiguos ("el error", "dinero que he ganado...") y los falsos positivos a mensajes legítimos con vocabulario inusual — fallos esperables y de bajo impacto.

5. Discusión

Trade-off Precision/Recall. La calibración del threshold materializa la decisión de diseño del PEAS: priorizar la detección de spam. Pasar de 0.5 a 0.189 sube el Recall a 98.1 %, dejando pasar solo el 1.9 % del spam, a costa de bloquear algunos *ham*. En un filtro real esto se mitiga enviando los positivos a una carpeta de cuarentena revisable, no eliminándolos.

Generalización cross-lingual. Que el rendimiento en español (F1 0.946) sea casi idéntico al inglés (F1 0.959) confirma que el enfoque es transferible entre idiomas. El reemplazo de URLs y números por *tokens* universales contribuye a esta robustez, pues esas señales son independientes del idioma.

Sobre los char n-grams. Aunque el modelo de *char n-grams* fue diseñado para ser robusto a errores ortográficos típicos del spam, el `GridSearchCV` terminó prefiriendo unigramas de palabra. Esto sugiere que, en este corpus ya balanceado y limpio, la señal a nivel de palabra es suficiente; los char n-grams aportarían más valor frente a ofuscación agresiva ("*V1AGR4*", "*g4n4 din3ro*").

Ausencia de overfitting. Las brechas pequeñas en cross-validation y la curva de aprendizaje (F1 train 0.989 vs validación 0.959, brecha 0.030) indican que el modelo generaliza y que añadir más datos seguiría ayudando marginalmente.

Limitaciones. (i) El dataset español es en gran parte *traducción automática* del inglés, no spam español nativo; (ii) el corpus es de SMS/mensajes cortos, no correos largos con HTML; (iii) el modelo es estático: el spam evoluciona y requeriría reentrenamiento periódico.

6. Conclusiones

1. Se construyó un detector de spam **bilingüe** end-to-end que alcanza **F1 = 0.952** y **Accuracy = 0.962** en el conjunto de prueba, superando las metas planteadas en la fase de planeación.
2. La **Regresión Logística con TF-IDF** fue el mejor modelo; el tuning de hiperparámetros y la **calibración del threshold** ($t^* = 0.189$) elevaron el Recall sobre spam al **98.1 %**, alineado con el objetivo de seguridad.
3. El sistema **generaliza entre idiomas** (F1 EN 0.959 / ES 0.946) y no presenta sobreajuste, validado con cross-validation y curva de aprendizaje.
4. Se entregó un **sistema interactivo** (ipywidgets + Gradio) que clasifica mensajes nuevos en vivo, y un modelo serializado reutilizable.

Trabajo futuro: incorporar spam español nativo, probar modelos *transformer* multilingües (p. ej. mBERT), y desplegar el filtro con reentrenamiento incremental.

7. Referencias

Almeida, T. A., & Gómez Hidalgo, J. M. (2011). *Contributions to the study of SMS spam filtering: New collection and results*. En Proceedings of the 11th ACM Symposium on Document Engineering (DocEng '11) (pp. 259–262). ACM.

Metsis, V., Androutsopoulos, I., & Paliouras, G. (2006). *Spam filtering with Naive Bayes — Which Naive Bayes?* En Proceedings of the 3rd Conference on Email and Anti-Spam (CEAS).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.^a ed.). Pearson.

Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.